Python Engineering - Week 03

Week 03 - Modules and coordinates

`01_python_bootcamp/week_03_modules_and_coordinates/`

How to use this document

This PDF is a review index for the week. Each class summarizes **the problem, concepts and syntax/functions introduced**. If a concept becomes fuzzy, locate it here and return to the original exercise.

class_09_modules_imports

EX17 - Vertical motion split into modules

Module: Reusable .py file containing functions, classes or constants.

Architecture: vertical_motion.py, data_processing.py and main.py with separate responsibilities.

Imports: Each module directly imports what it uses.

main(): Orchestrates the application; calculation logic stays in reusable modules.

__**name**__**:** if __name__ == '__main__': runs main only when executed directly.

Design: process_trajectory() returns data and avoids internal printing for reuse/testing.

class_10_advanced_functions

EX18 - Advanced function arguments

Arguments: positional, keyword and default arguments.

***args:** Collects variable positional arguments into a tuple.

****kwargs:** Collects variable keyword arguments into a dict.

Unpacking: *sequence and **mapping when calling functions.

Functions as objects: A function can be passed without executing it: add versus add(...).

Key idea: A function signature defines how information enters; unpacking adapts structures to that signature.

class_11_coordinate_transforms

EX19 - Rotation matrices

Mathematics: 2D rotations and z-axis rotation extension in 3D.

NumPy: Matrix-vector multiplication with @.

Properties: R.T @ R = I, R^-1 = R.T for rotation matrices, det(R)=+1.

Validation: Norm preservation and np.allclose.

Composition: R(theta2) @ R(theta1) equals angle addition for the same axis.

Concepts: active rotation versus passive/change-of-frame convention.

class_12_engineering_toolkit

EX20 - Reusable vector toolkit

Functions: vector_norm(), unit_vector(), angle_between_vectors(), project_vector().

Rotations: rotation_matrix_2d_angle(), rotate_2d_vector(), check_rotation_matrix().

NumPy: np.linalg.norm, np.dot, np.clip, np.arccos, np.rad2deg, np.eye, np.linalg.det, np.allclose.

Numerics: Clip cosine before arccos to protect the [-1, 1] domain.

Validation: A proper rotation needs orthogonality and determinant +1; a reflection may be orthogonal with det -1.

Architecture: engineering_toolkit.py contains tools; main.py creates data, calls tools and presents results.

Quick concept index

Concept	Where to find it
Module: Reusable .py file containing functions, classes or constants.	class_09_modules_imports
Architecture: vertical_motion.py, data_processing.py and main.py with separate responsibilities.	class_09_modules_imports
Imports: Each module directly imports what it uses.	class_09_modules_imports
main(): Orchestrates the application; calculation logic stays in reusable modules.	class_09_modules_imports
__name__: if __name__ == '__main__': runs main only when executed directly.	class_09_modules_imports
Design: process_trajectory() returns data and avoids internal printing for reuse/testing.	class_09_modules_imports
Arguments: positional, keyword and default arguments.	class_10_advanced_functions
*args: Collects variable positional arguments into a tuple.	class_10_advanced_functions
**kwargs: Collects variable keyword arguments into a dict.	class_10_advanced_functions
Unpacking: *sequence and **mapping when calling functions.	class_10_advanced_functions
Functions as objects: A function can be passed without executing it: add versus add(...).	class_10_advanced_functions
Key idea: A function signature defines how information enters; unpacking adapts structures to that signature.	class_10_advanced_functions
Mathematics: 2D rotations and z-axis rotation extension in 3D.	class_11_coordinate_transforms
NumPy: Matrix-vector multiplication with @.	class_11_coordinate_transforms
Properties: R.T @ R = I, R^-1 = R.T for rotation matrices, det(R)=+1.	class_11_coordinate_transforms
Validation: Norm preservation and np.allclose.	class_11_coordinate_transforms
Composition: R(theta2) @ R(theta1) equals angle addition for the same axis.	class_11_coordinate_transforms
Concepts: active rotation versus passive/change-of-frame convention.	class_11_coordinate_transforms
Functions: vector_norm(), unit_vector(), angle_between_vectors(), project_vector().	class_12_engineering_toolkit

Review rule

First locate **the concept**; then open that class PDF; finally run or modify the exercise. Document + code is the review mechanism for this course.